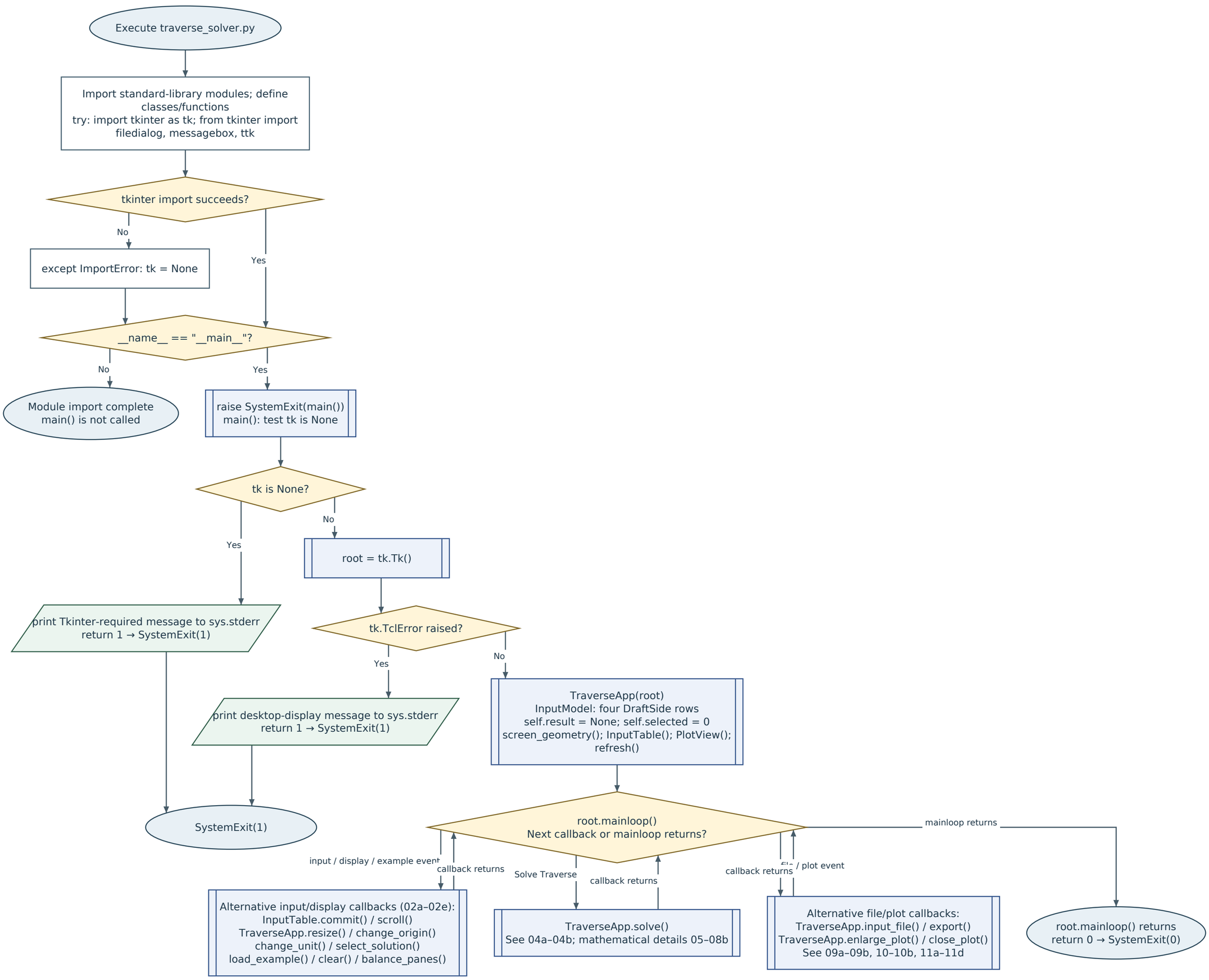
01 Program entry and Tkinter event loop

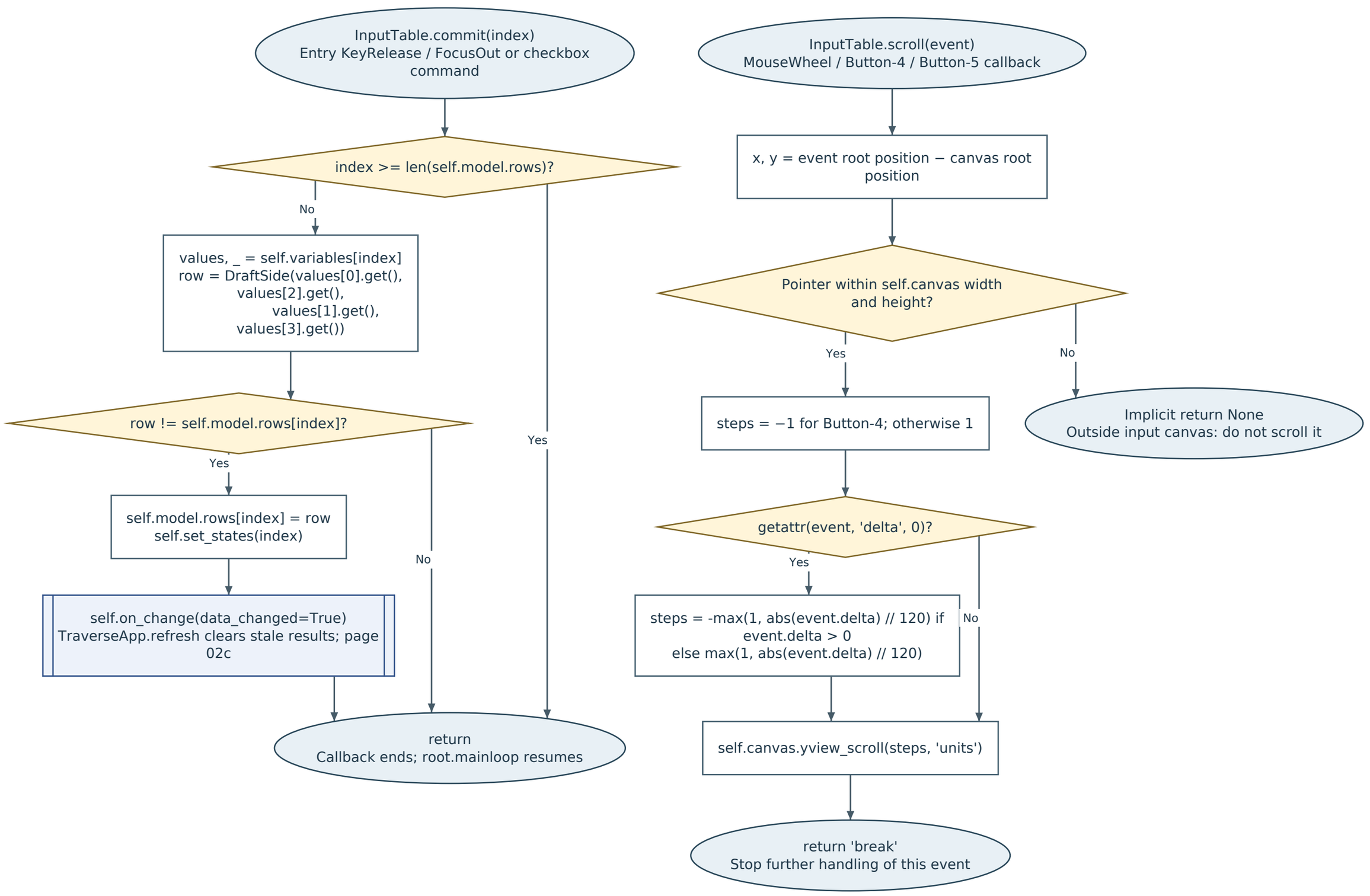
Read from main() and TraverseApp.__init__(). Callback branches are alternatives, not a sequential workflow. Page identifiers refer to the linked detail diagrams.



Callback return paths rejoin root.mainloop().
Root window closure uses Tkinter default behavior; no custom root close handler is defined.
PlotView initialization catches Exception internally (10). Other unhandled callback exceptions are not converted to success by these diagrams.

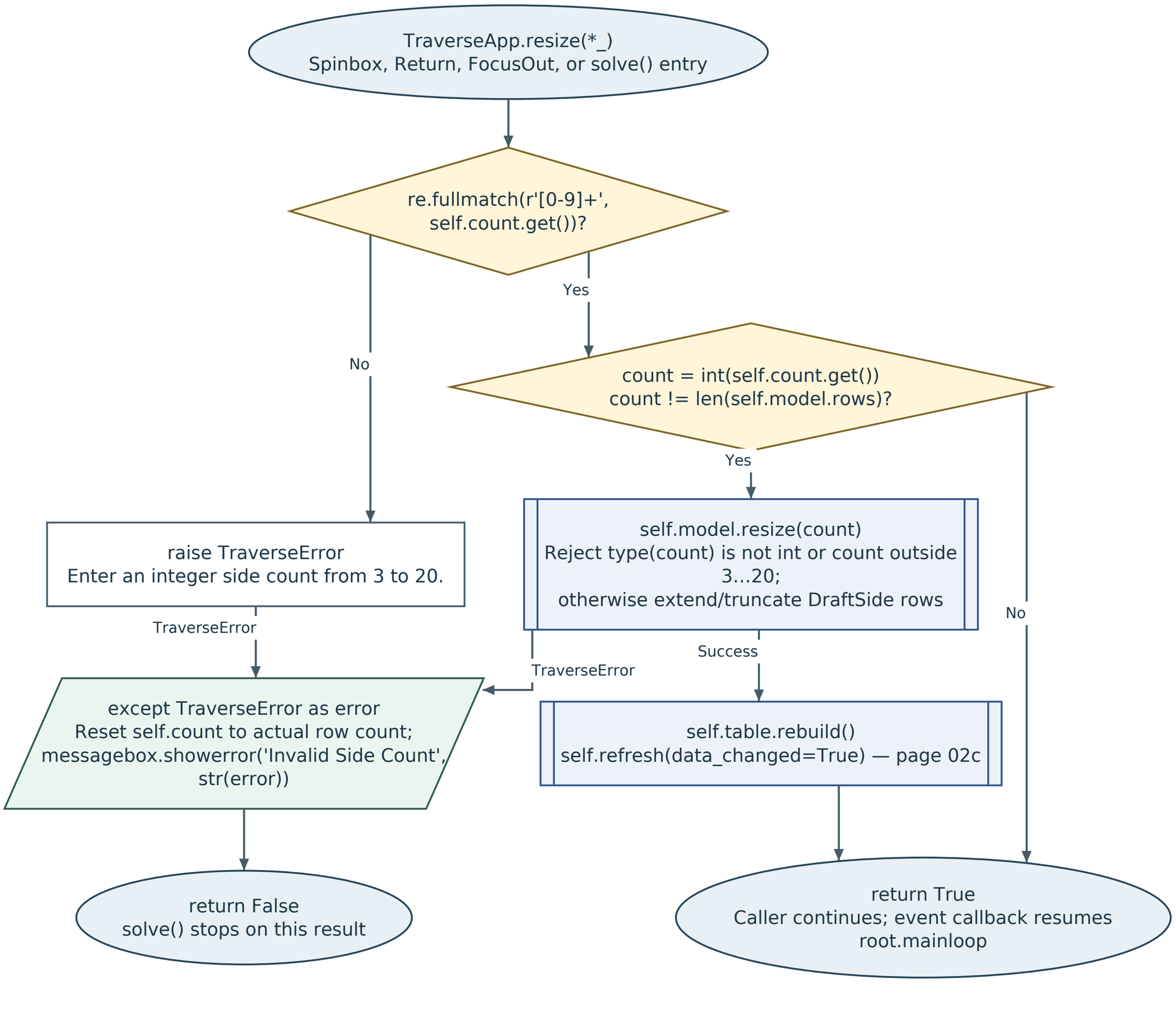
02a Input-table edits and scrolling

InputTable.commit and InputTable.scroll; widget variables commit into InputModel.rows.



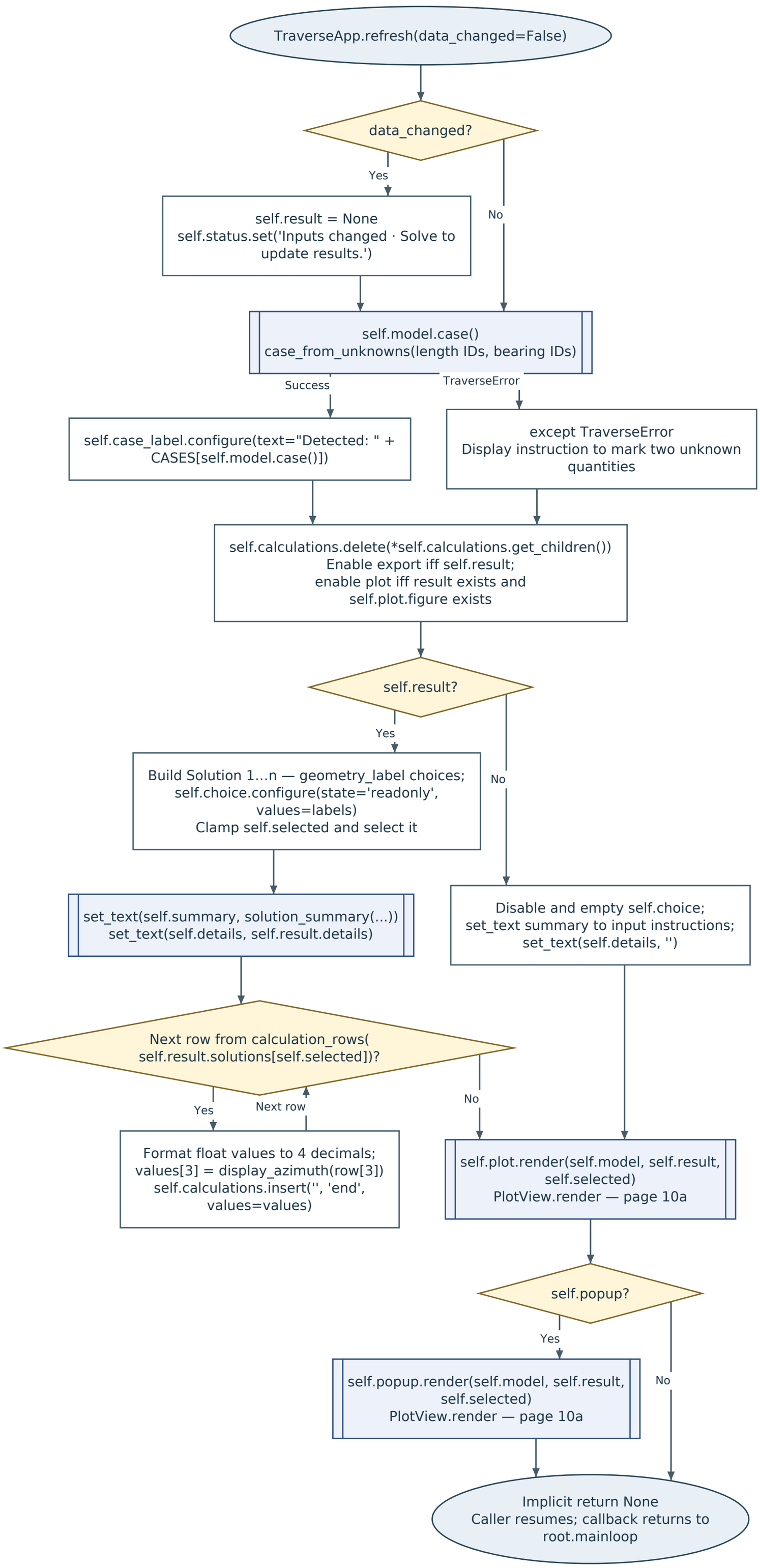
02b Side count changes and validation

TraverseApp.resize(): spinner arrows, Return, FocusOut, or the first step of solve().



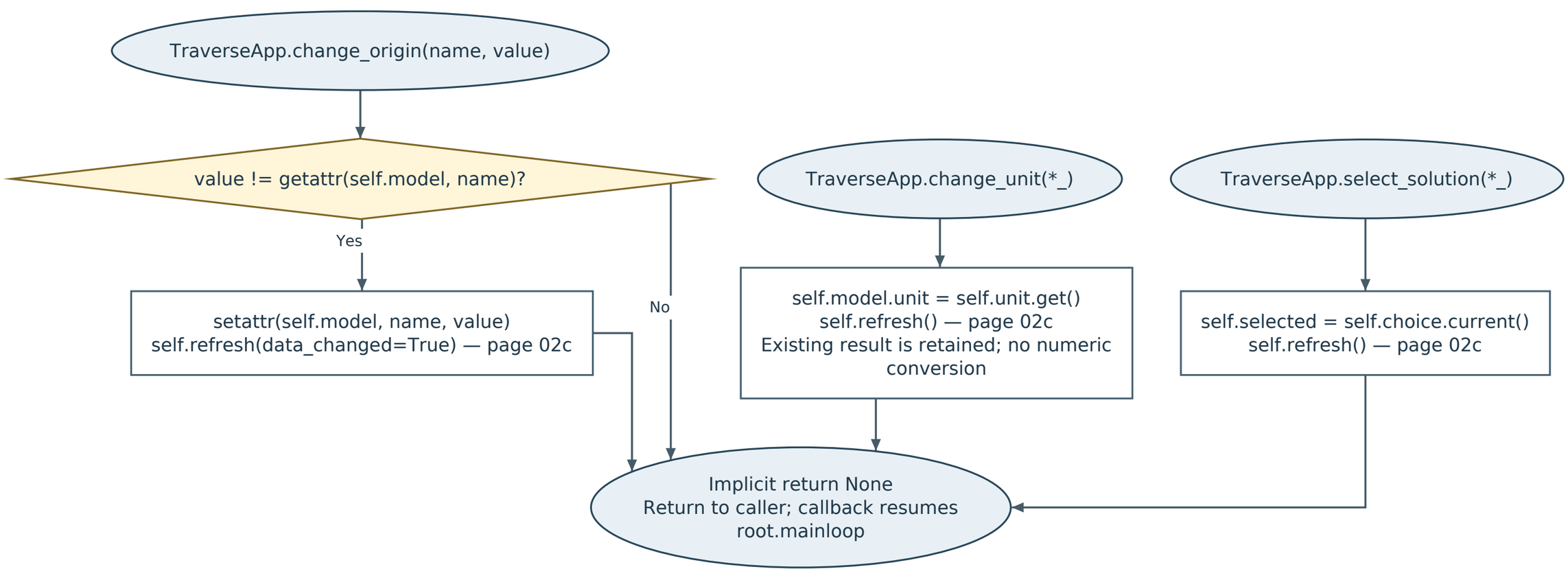
02c Central view refresh

TraverseApp.refresh uses InputModel plus SolveResult; no unsolved-traverse preview is drawn.



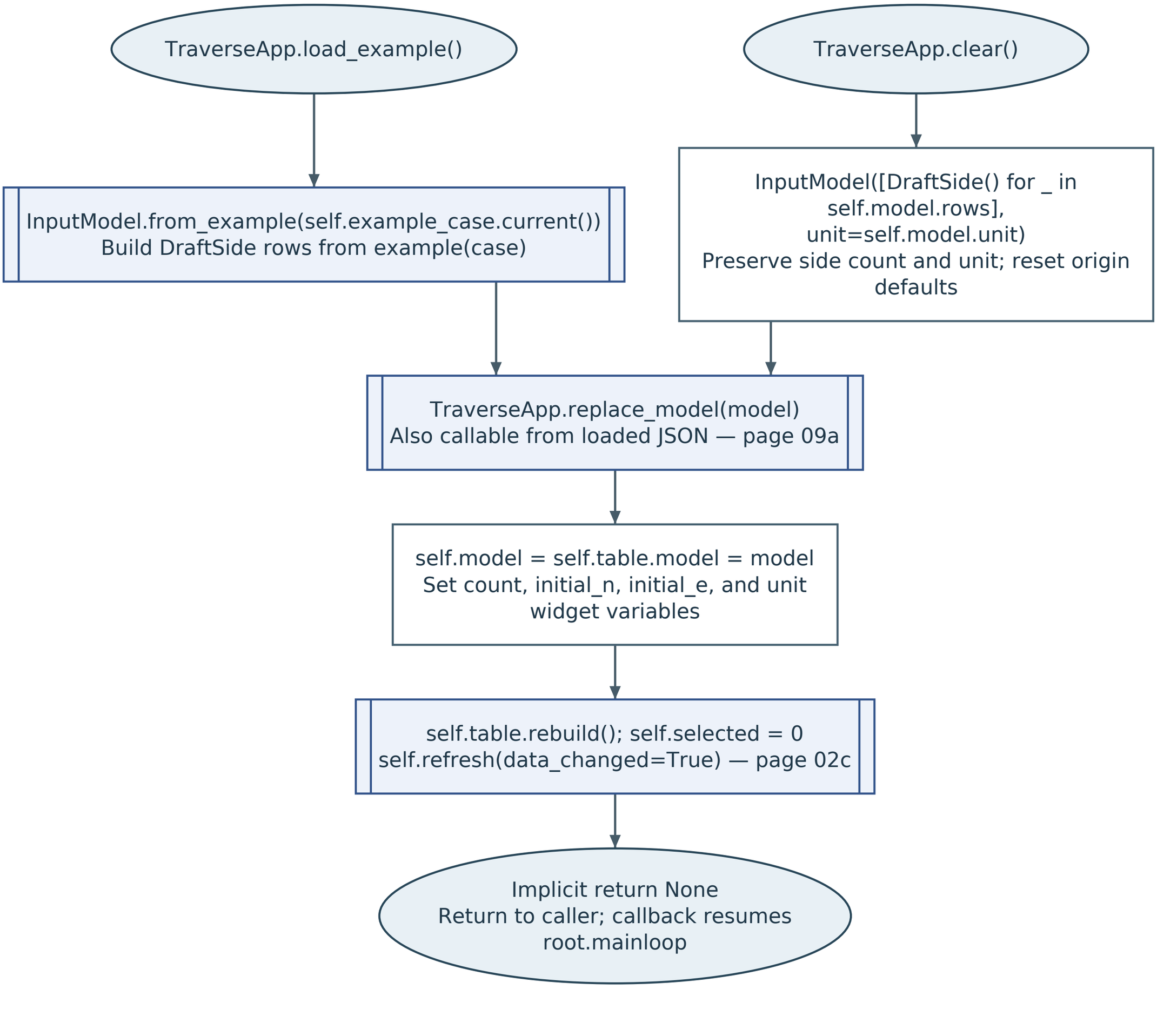
02d Origin, unit, and solution-selection callbacks

Separate callback entries. Only data changes invalidate self.result; refresh() is detailed on 02c.



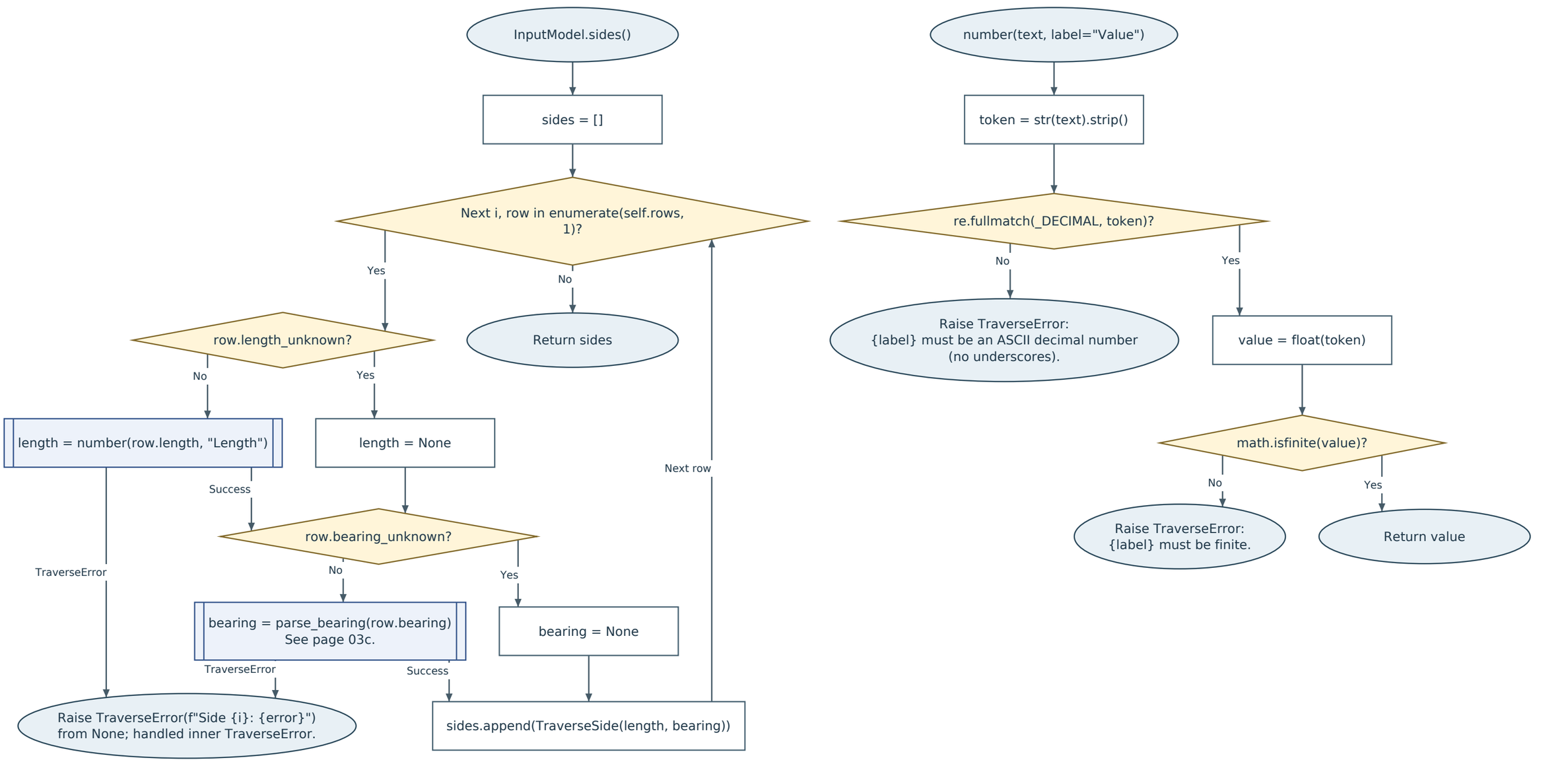
02e Example, clear, and model replacement

Load Example and Clear construct an InputModel, then TraverseApp.replace_model() refreshes widgets and invalidates old results.



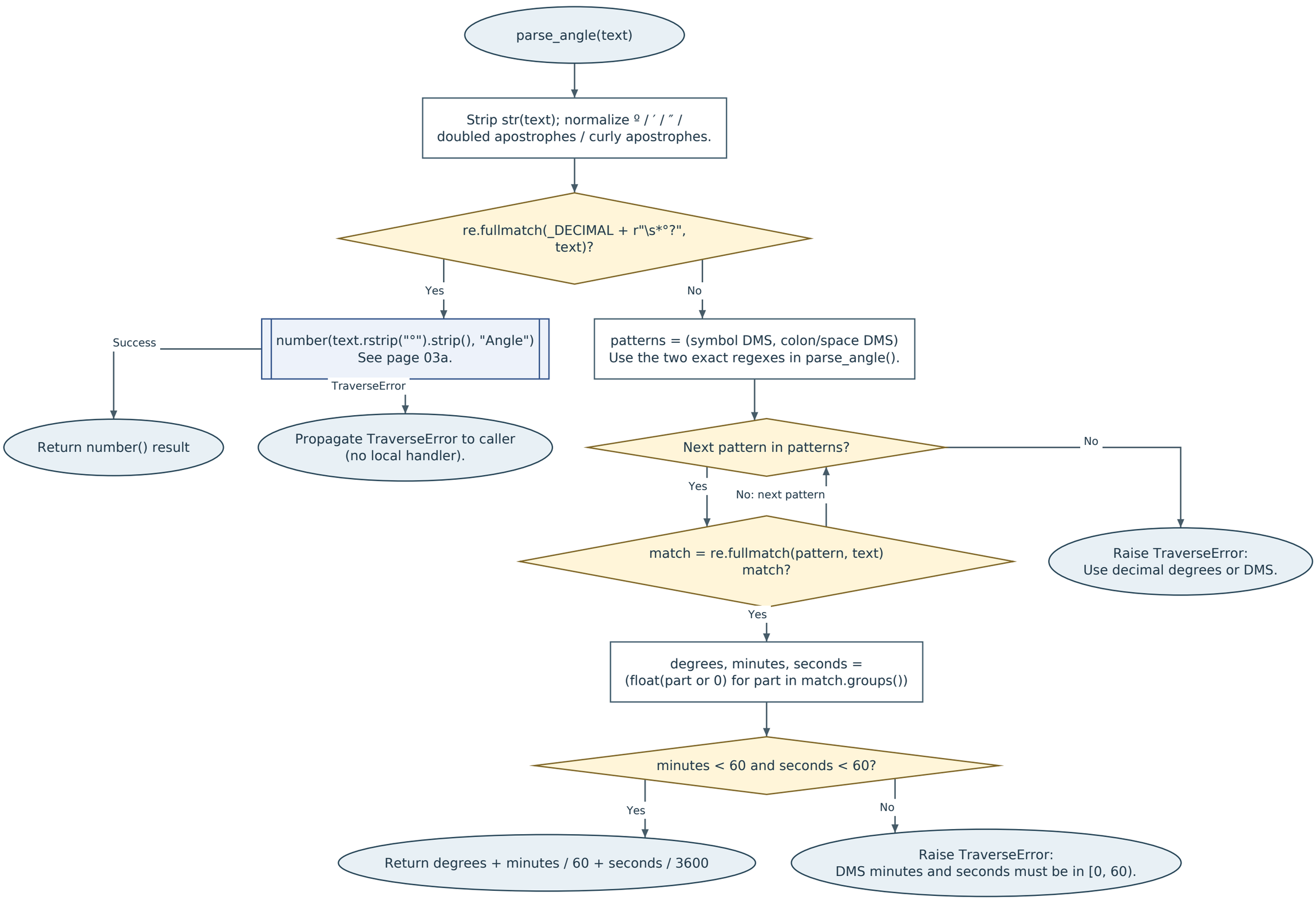
03a Parse input rows and decimal numbers

InputModel.sides() wraps TraverseError with its side number; number() accepts ASCII decimal syntax only.



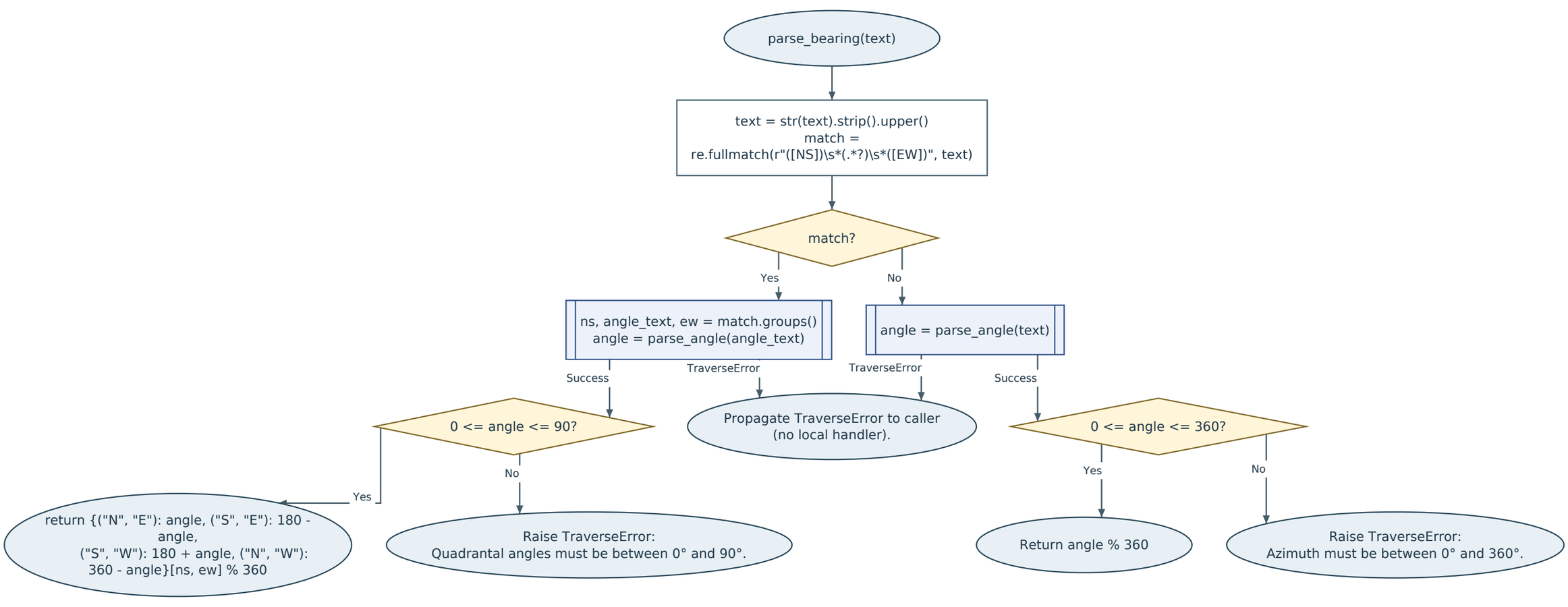
03b Angle parsing: decimal or DMS

parse_angle(text): normalization and the two DMS pattern attempts. Errors propagate to the caller.



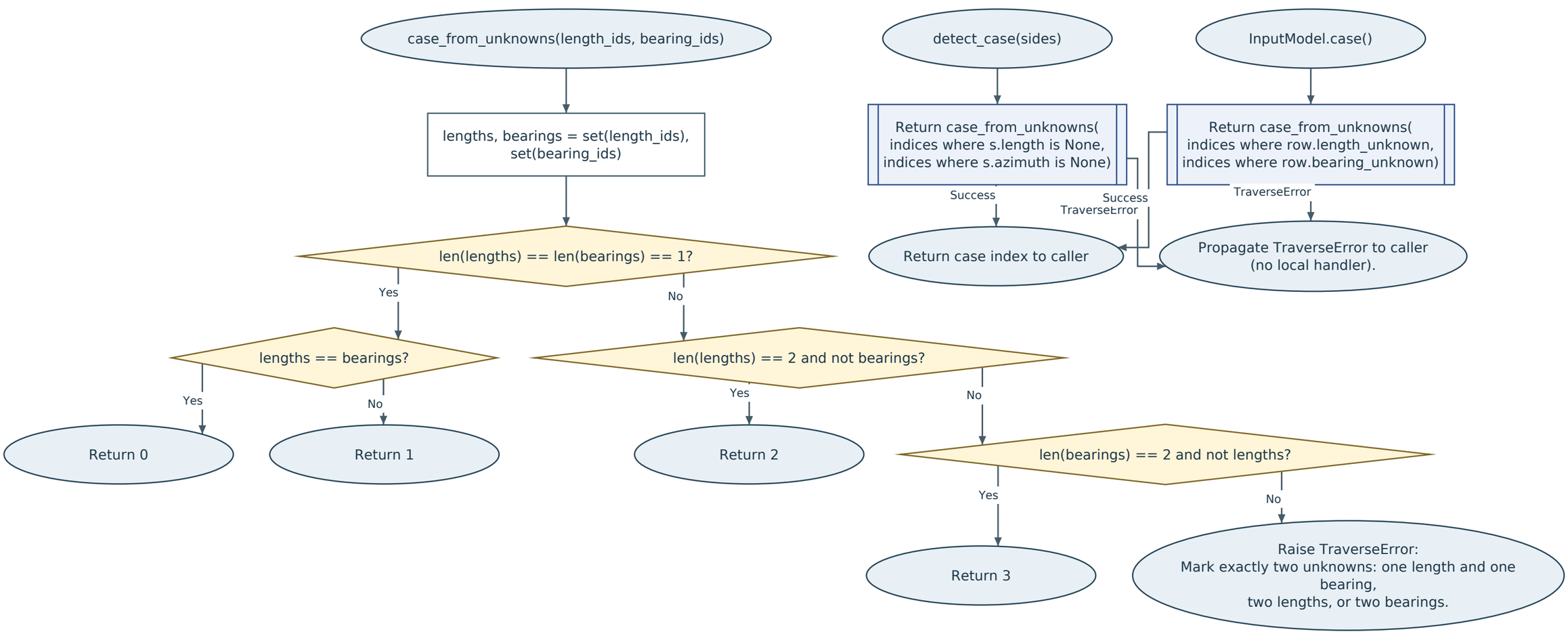
03c Bearing parsing: quadrantal or whole-circle

parse_bearing(text) calls parse_angle(), detailed on 03b; number() is detailed on 03a.



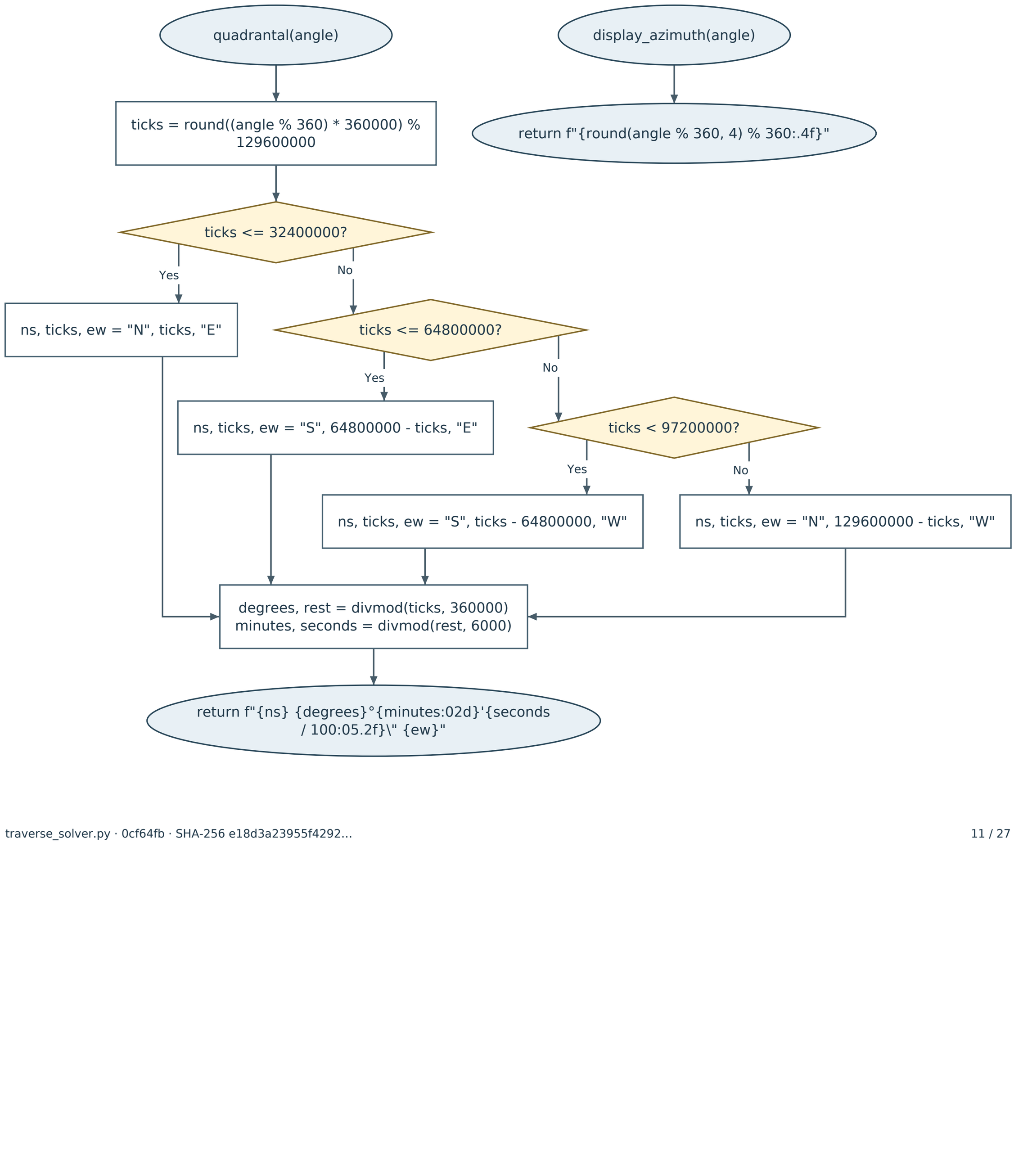
03d Detect the omitted-measurement case

CASES index mapping (exact source constants):
0: Case I: Length & Bearing of One Side Omitted
1: Case II: Length of One Side & Bearing of Another Omitted
2: Case III: Length of Two Sides Omitted
3: Case IV: Bearing of Two Sides Omitted



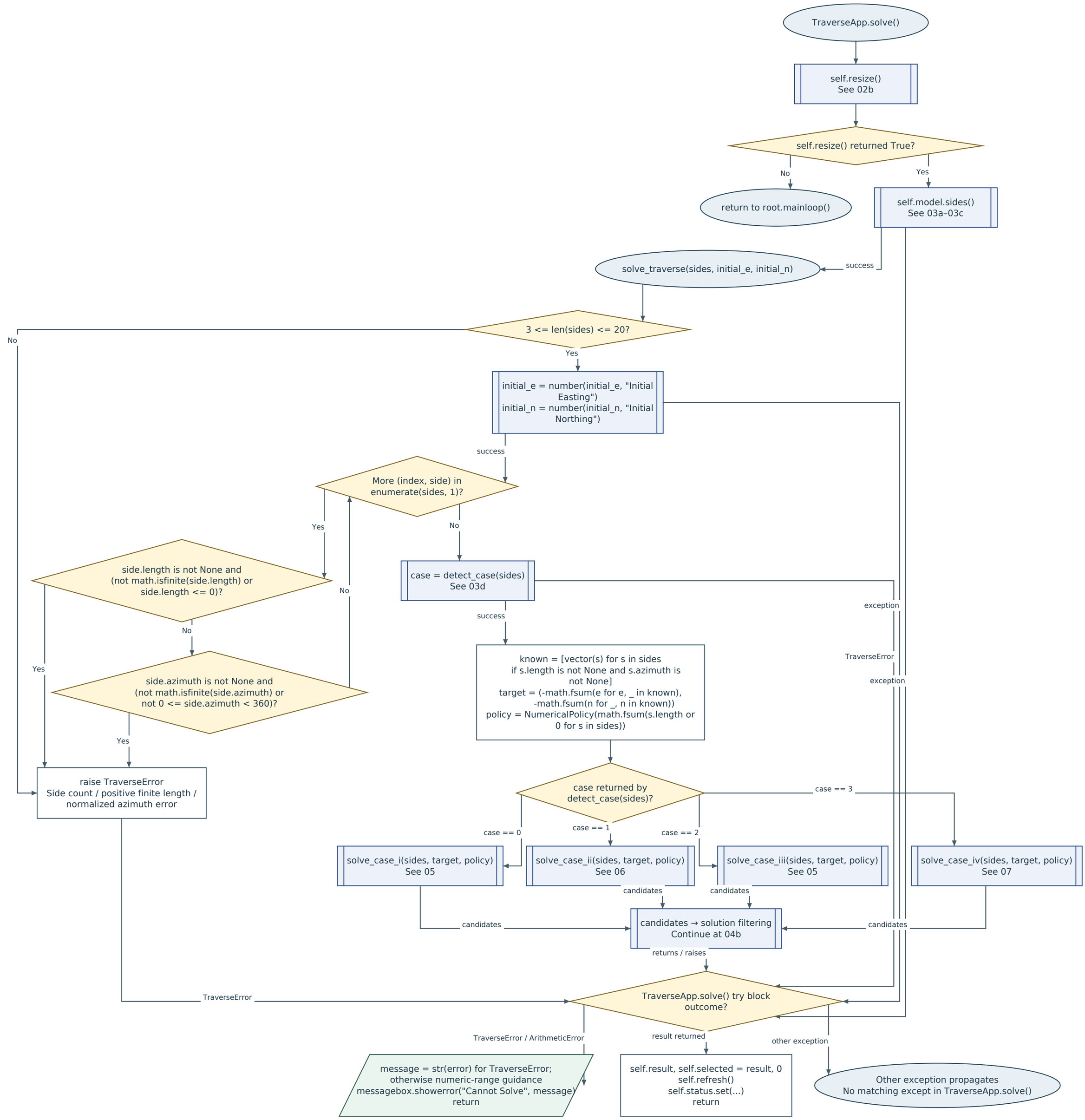
03e Format quadrantal and azimuth bearings

quadrantal() rounds before choosing the quadrant; display_azimuth() rounds to four decimal places and normalizes the displayed value.



04a Solve callback, validation, and solver dispatch

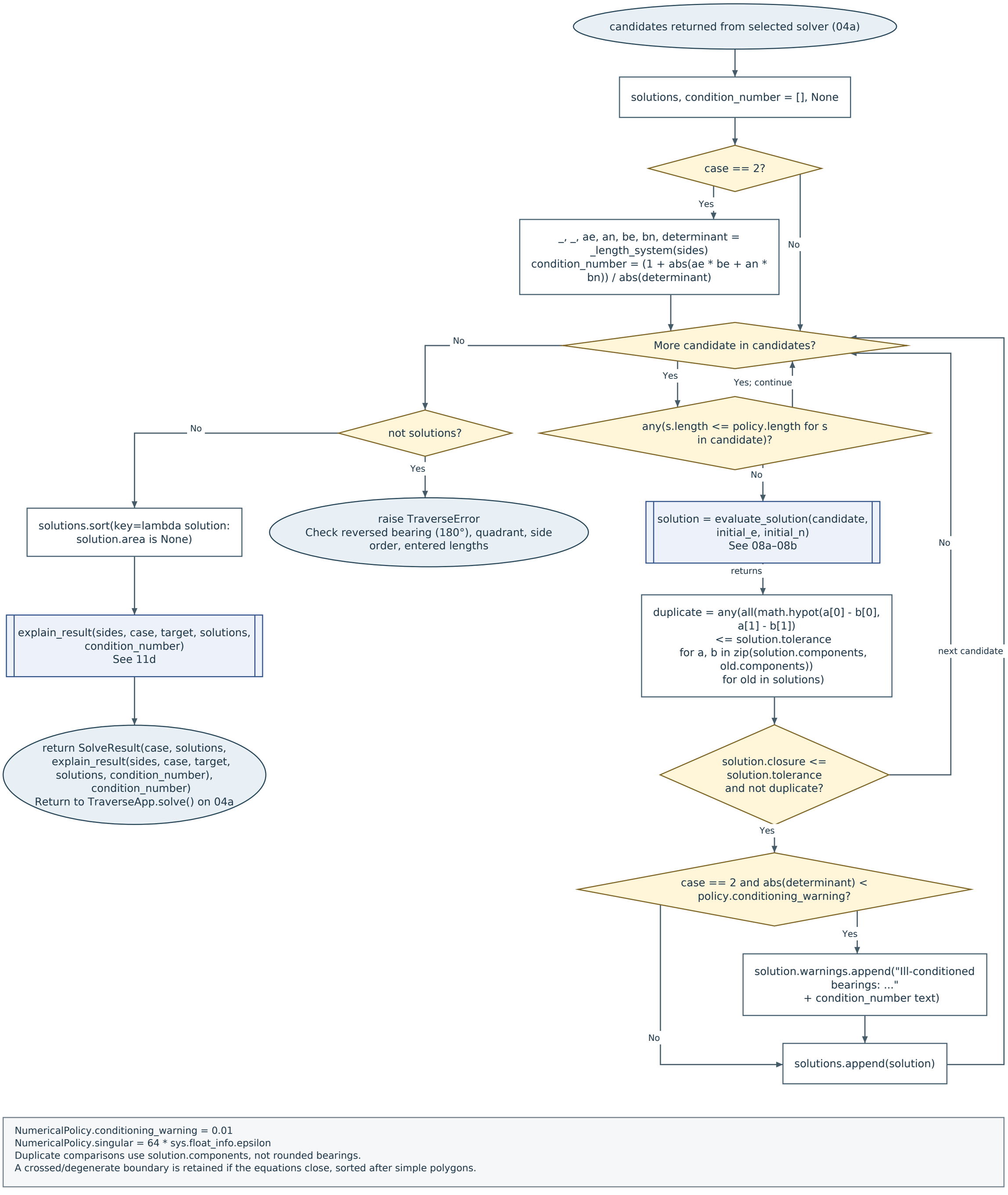
TraverseApp.solve() calls resize() before its try block. Input parsing is detailed in 03a–03b; case detection in 03d. A raised TraverseError or ArithmeticError is caught by TraverseApp.solve().



Exceptions from InputModel.sides(), number(), detect_case(), case solvers, or candidate evaluation all propagate to the same TraverseApp.solve() catch. Unmatched exceptions are not caught there.
CASES exact descriptions are listed on 03d. The tuple dispatch uses indices 0, 1, 2, 3.

04b Candidate acceptance, conditioning, and result return

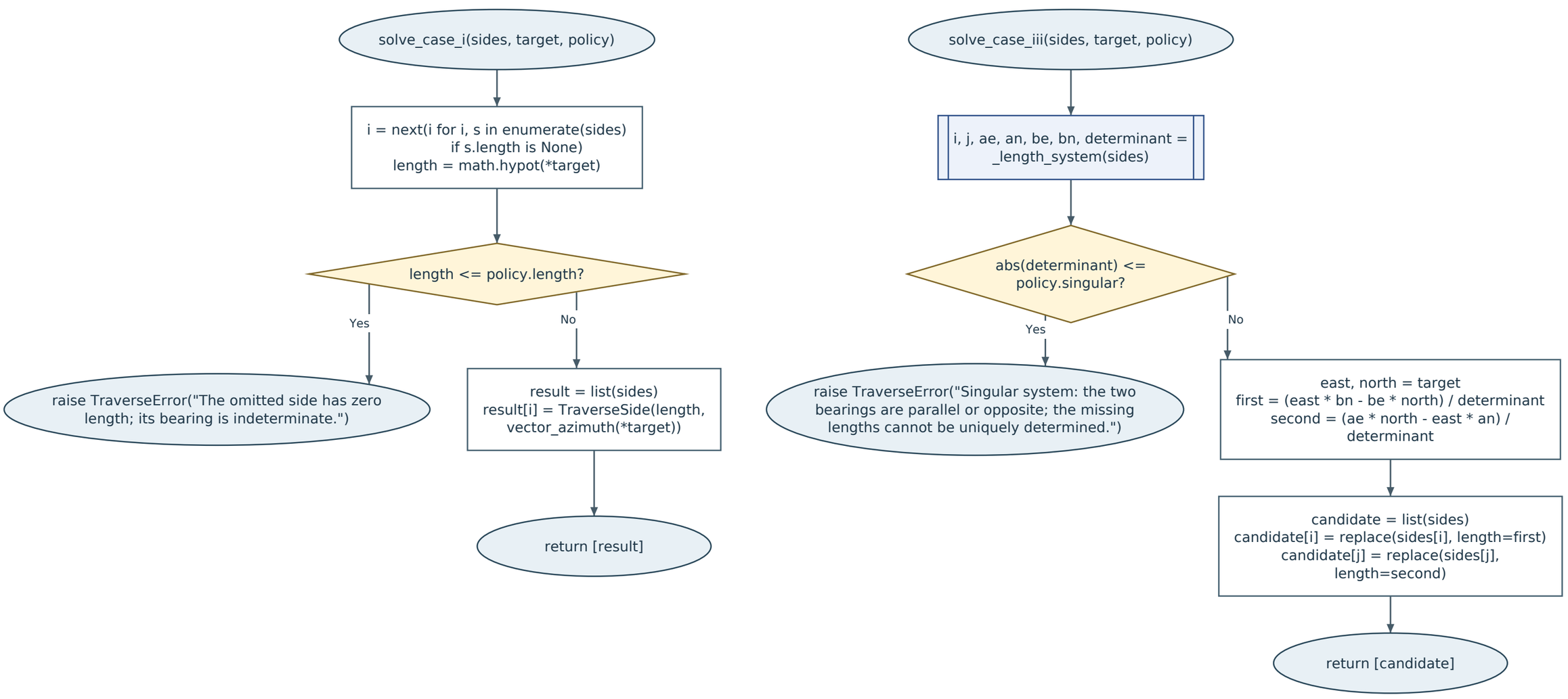
Continuation of solve_traverse() after the selected case function returns candidates. Return SolveResult to TraverseApp.solve() on 04a; raised errors propagate to that caller.



NumericalPolicy.conditioning_warning = 0.01
NumericalPolicy.singular = 64 * sys.float_info.epsilon
Duplicate comparisons use solution.components, not rounded bearings.
A crossed/degenerate boundary is retained if the equations close, sorted after simple polygons.

05 Cases I and III — omitted lengths

Case I: Length & Bearing of One Side Omitted • Case III: Length of Two Sides Omitted. Independent function entries; exceptions propagate through solve_traverse() to TraverseApp.solve().



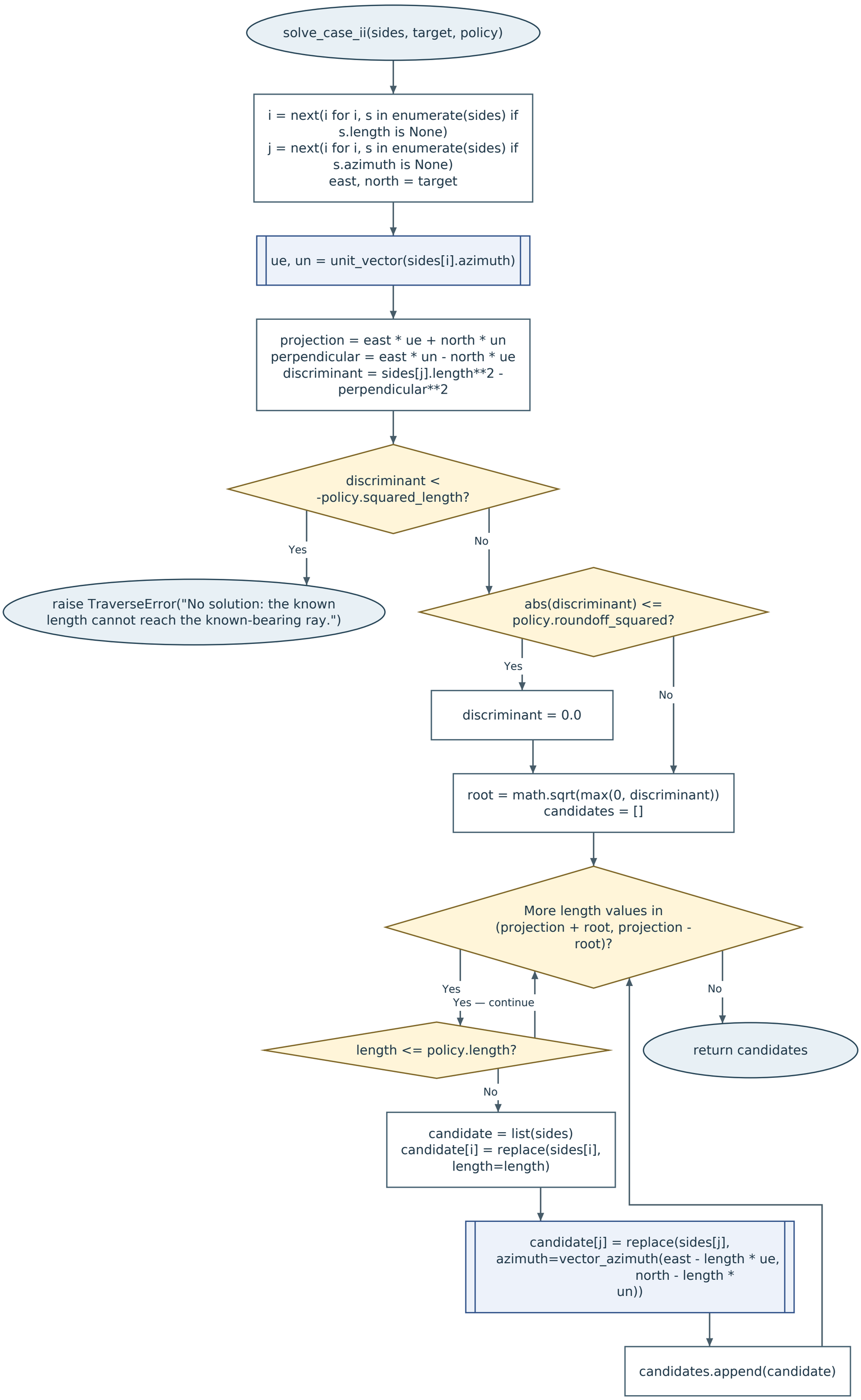

```
_length_system(sides):
i, j = [i for i, s in enumerate(sides) if s.length is None]
ae, an = unit_vector(sides[i].azimuth)
be, bn = unit_vector(sides[j].azimuth)
return i, j, ae, an, be, bn, ae * bn - be * an
```

```
NumericalPolicy.length: return self.scale * self.relative
relative: float = 1e-10
singular = 64 * sys.float_info.epsilon
Positive candidate lengths and closure are checked later in solve_traverse().
```

```
unit_vector(angle):
return math.sin(math.radians(angle)), math.cos(math.radians(angle))
vector_azimuth(east, north):
angle = math.degrees(math.atan2(east, north)) % 360
return 0.0 if angle == 360 else angle
```

06 Case II — omitted length and another bearing

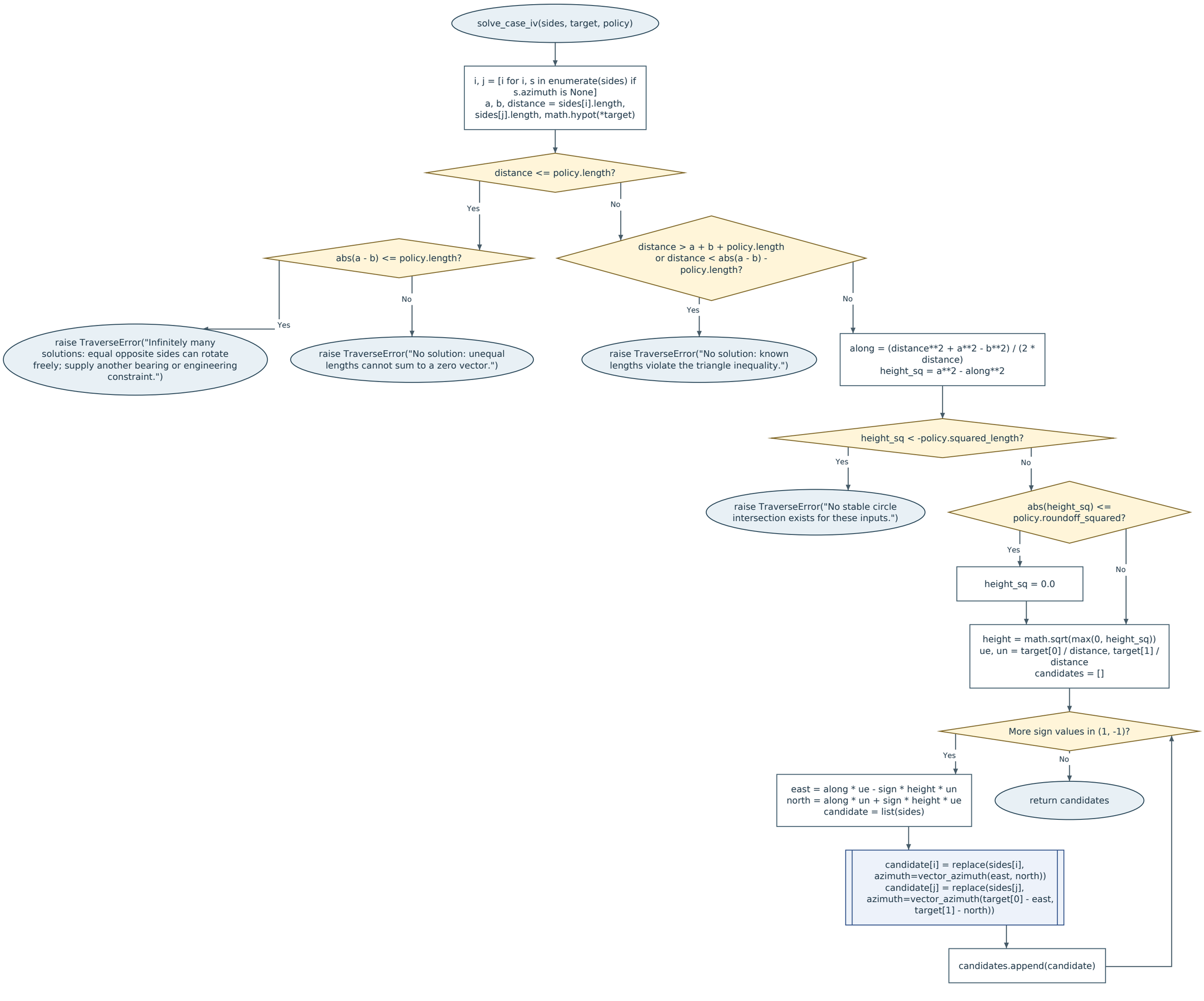
Case II: Length of One Side & Bearing of Another Omitted. Each positive quadratic root produces a candidate; solve_traverse() filters closure and duplicates.



NumericalPolicy.squared_length: return self.scale * self.length
NumericalPolicy.roundoff_squared:
return 64 * sys.float_info.epsilon * self.scale**2
An empty candidates list is returned if neither length is positive.

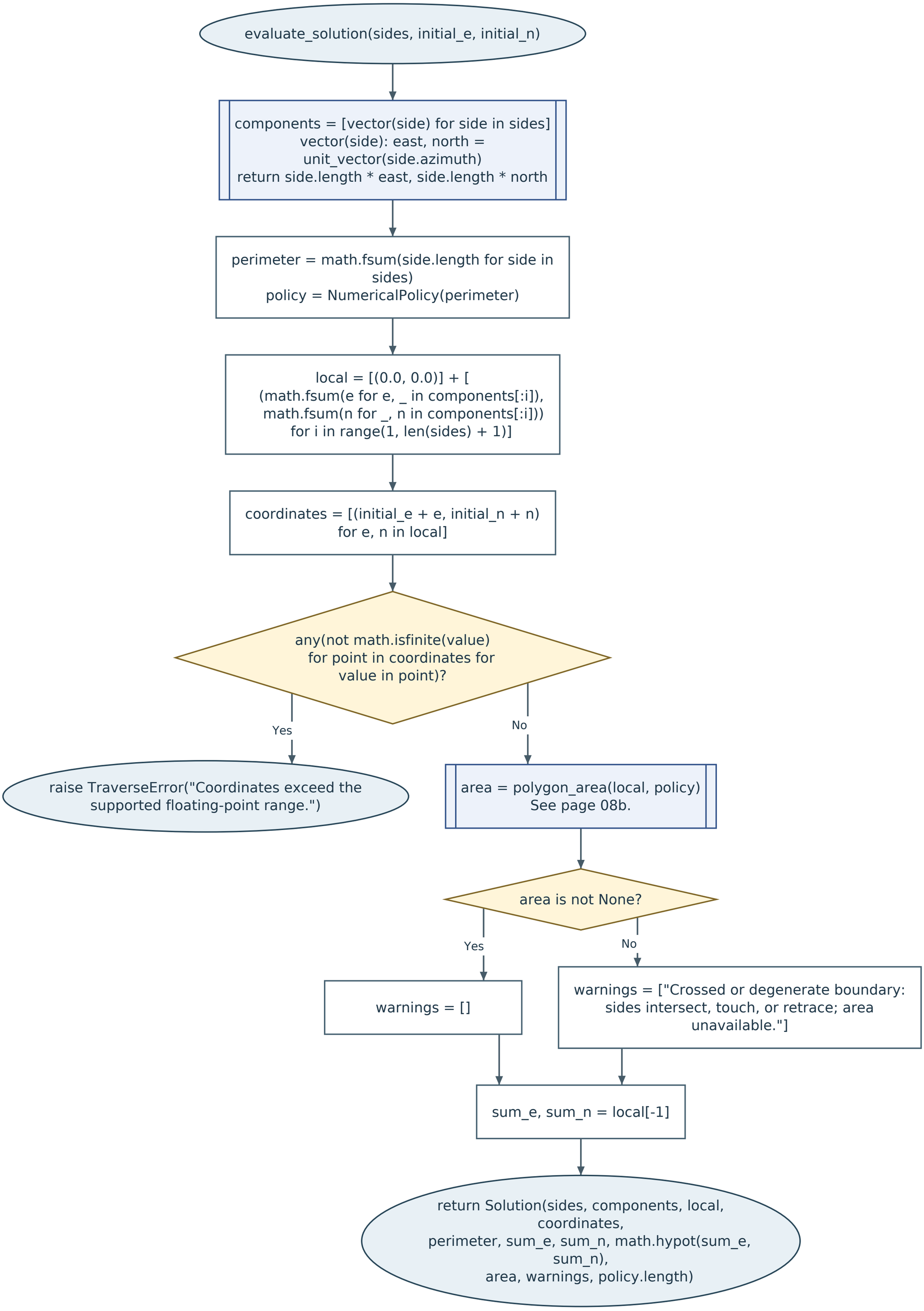
07 Case IV — two omitted bearings

Case IV: Bearing of Two Sides Omitted. Construct both sign choices; duplicate and crossed candidates are handled later by solve_traverse().



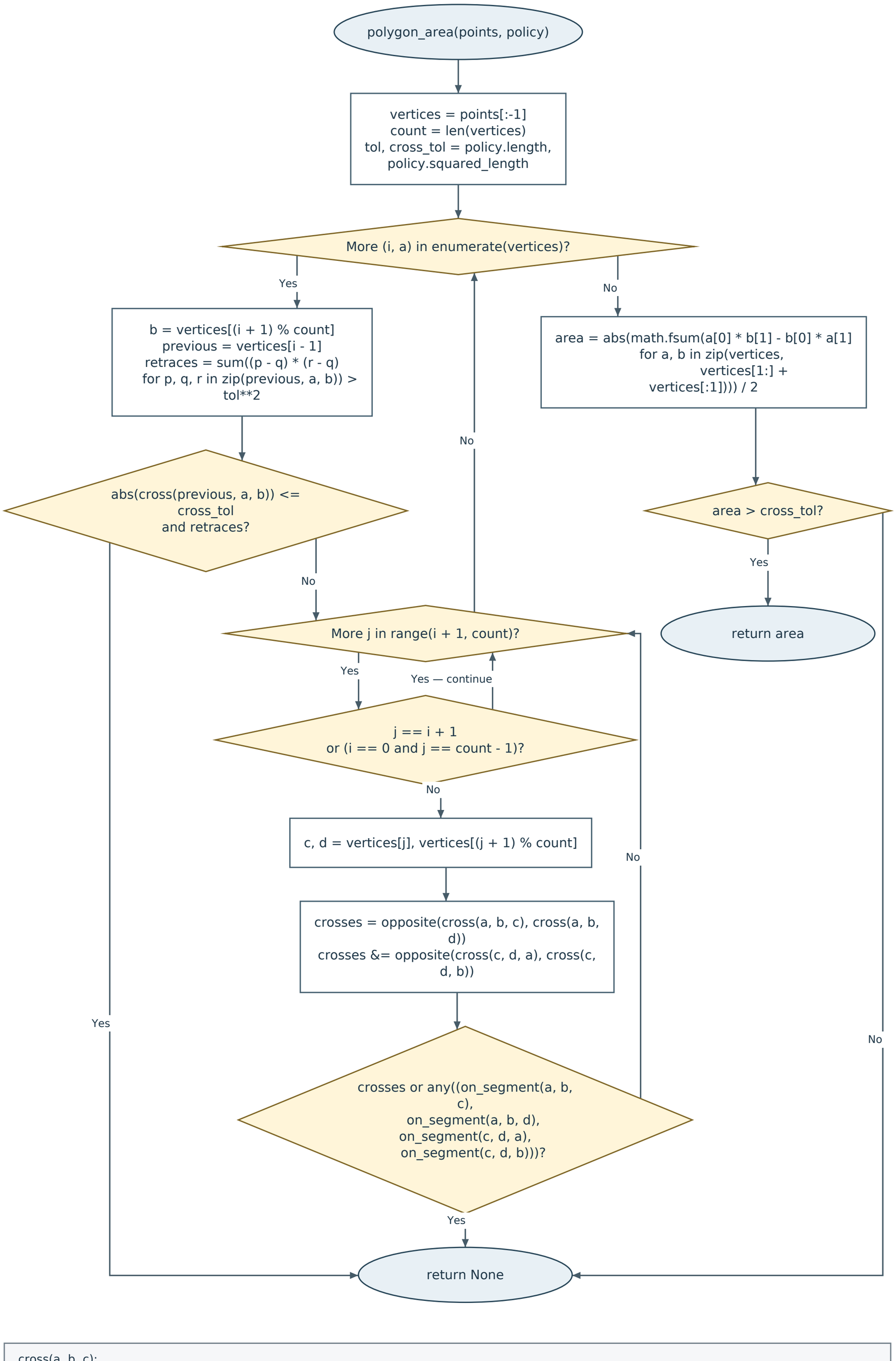
08a Evaluate one candidate

evaluate_solution() derives components and coordinates, rejects non-finite coordinates, calculates polygon area, and returns a Solution. It does not decide candidate acceptance.



08b Polygon boundary checks and shoelace area

polygon_area(points, policy): return None for adjacent retracing, non-neighbor intersections or touching, or area at/below cross_tol. Called with local coordinates by evaluate_solution().



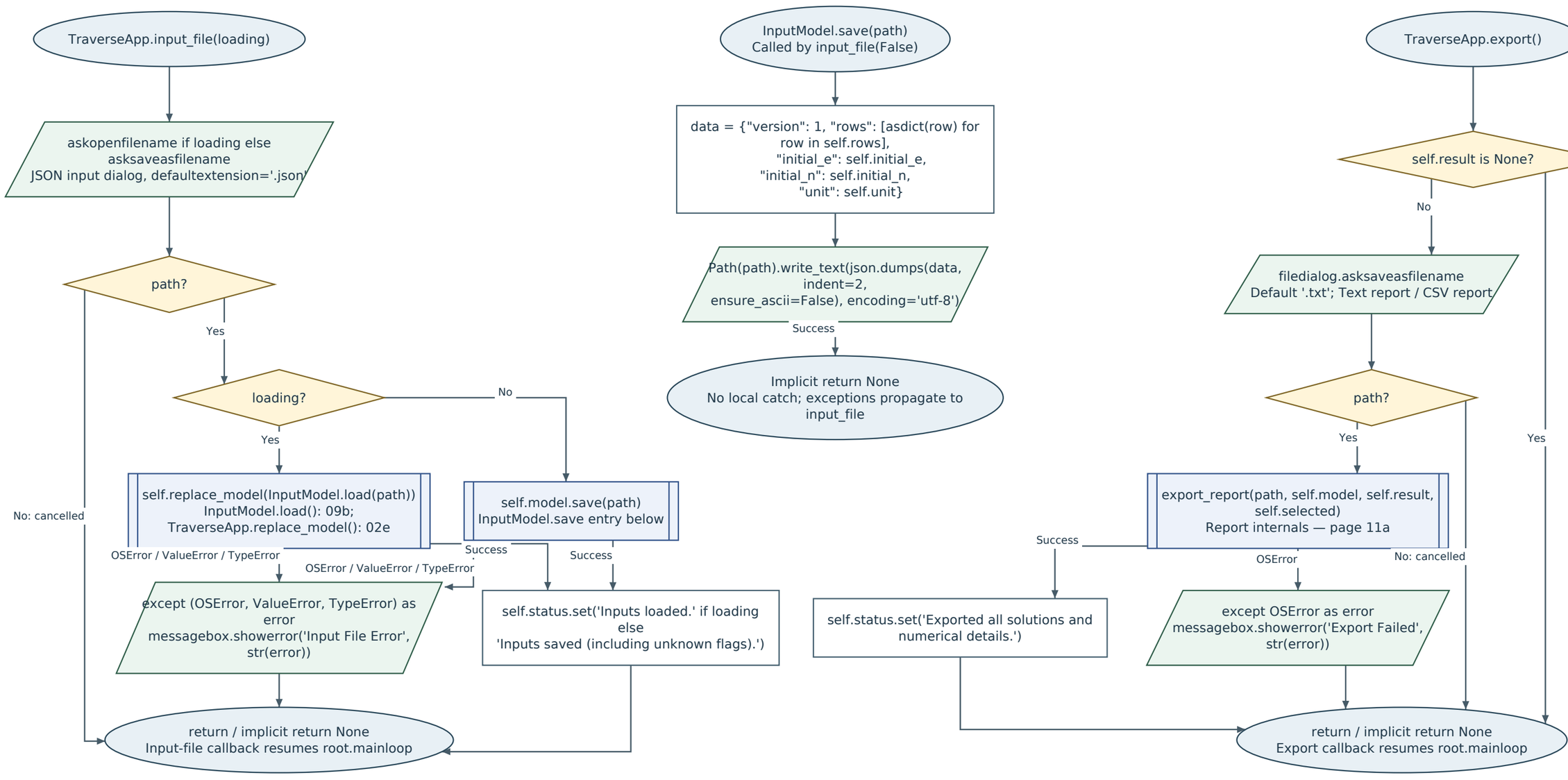

```
cross(a, b, c):
return (b[0] - a[0]) * (c[1] - a[1])
       - (b[1] - a[1]) * (c[0] - a[0])
```

```
opposite(a, b):
return (a > cross_tol and b < -cross_tol)
    or (a < -cross_tol and b > cross_tol)
```

```
on_segment(a, b, p):
return (abs(cross(a, b, p)) <= cross_tol
    and min(a[0], b[0]) - tol <= p[0] <= max(a[0], b[0]) + tol
    and min(a[1], b[1]) - tol <= p[1] <= max(a[1], b[1]) + tol)
```

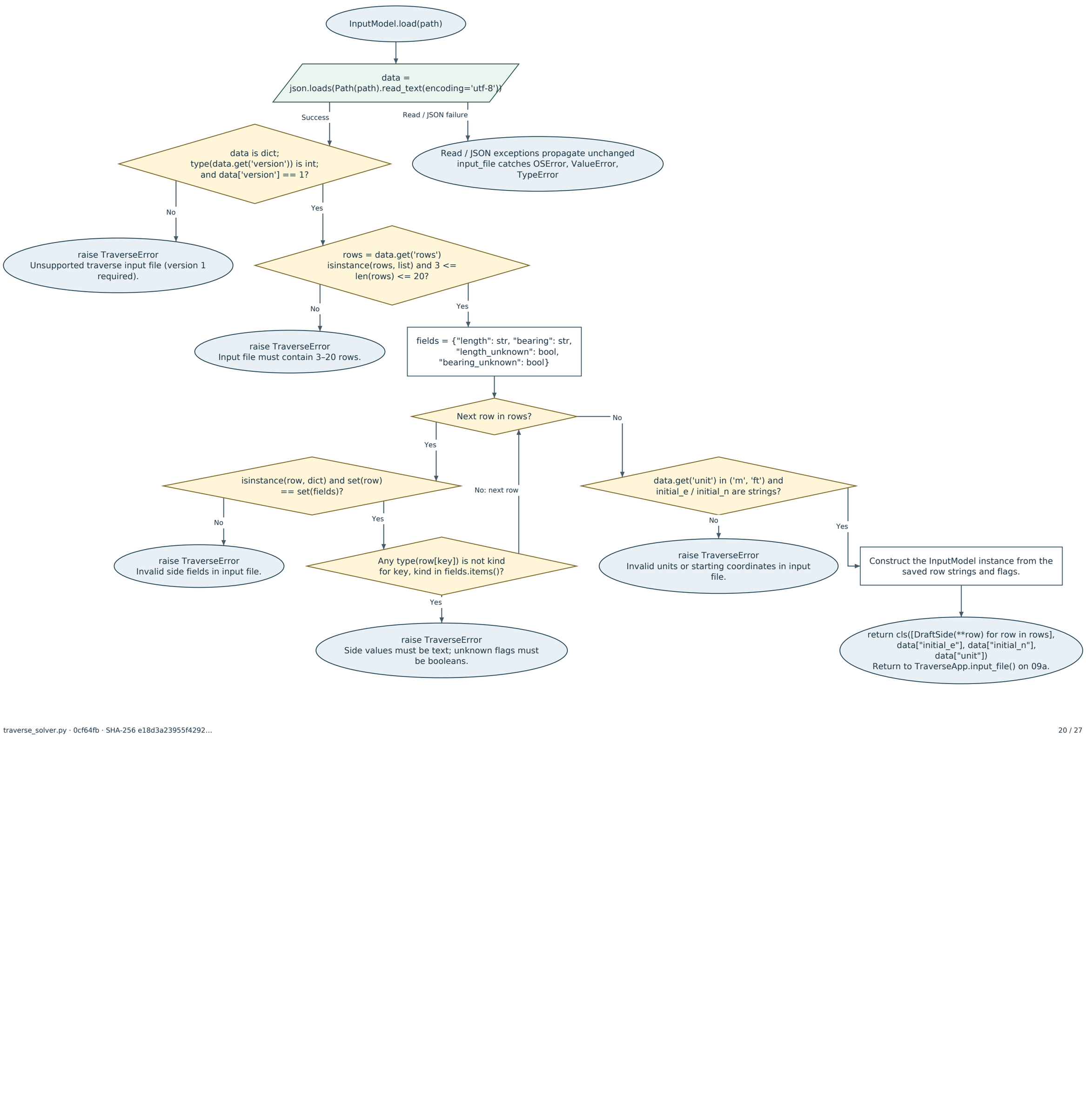
09a Input files and result export

Dialog cancellation returns without writing; only the exceptions caught by the actual callbacks recover to the GUI.



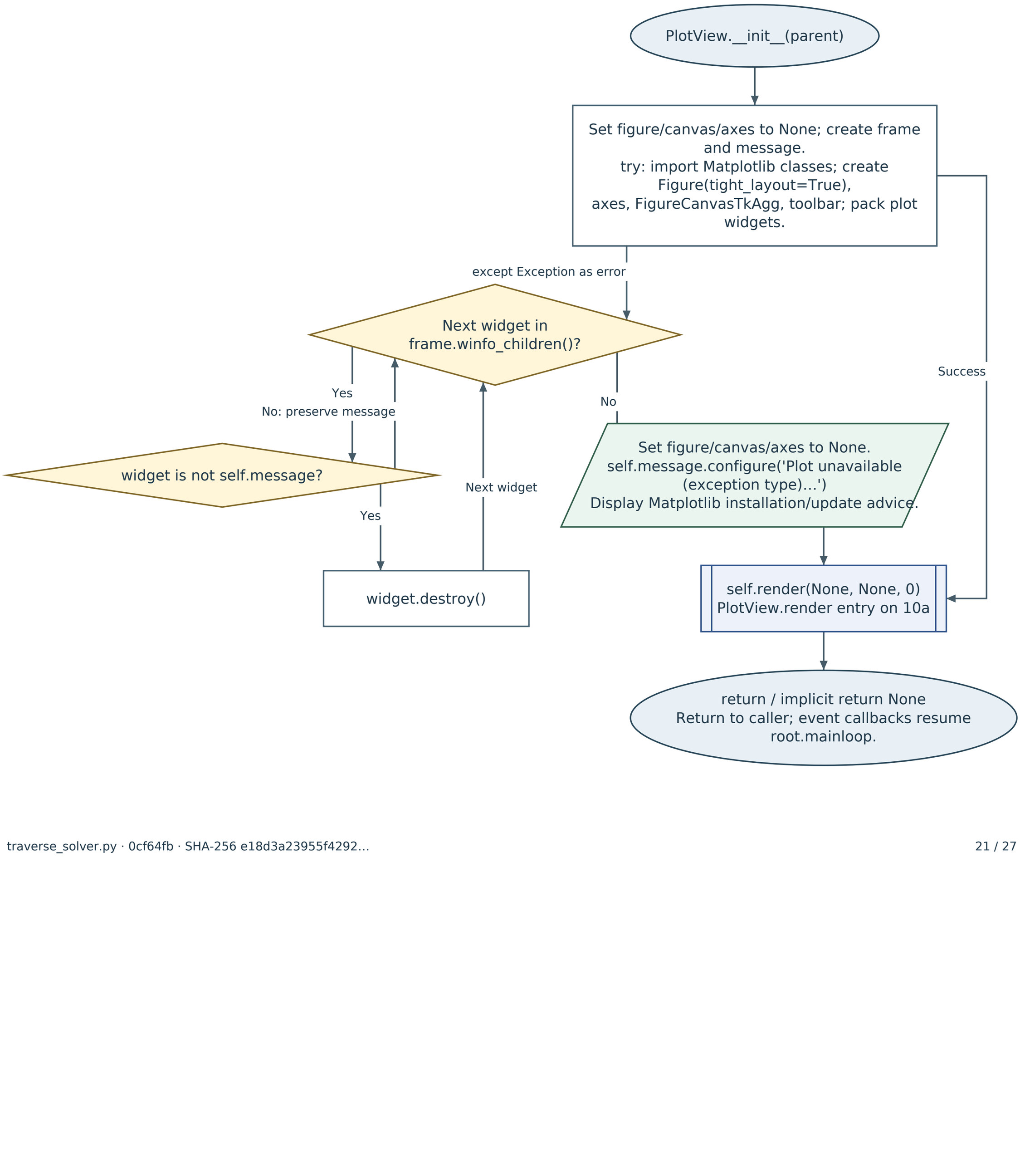
09b JSON input validation

InputModel.load validates file structure and field types; numeric and bearing text is parsed later when solving.



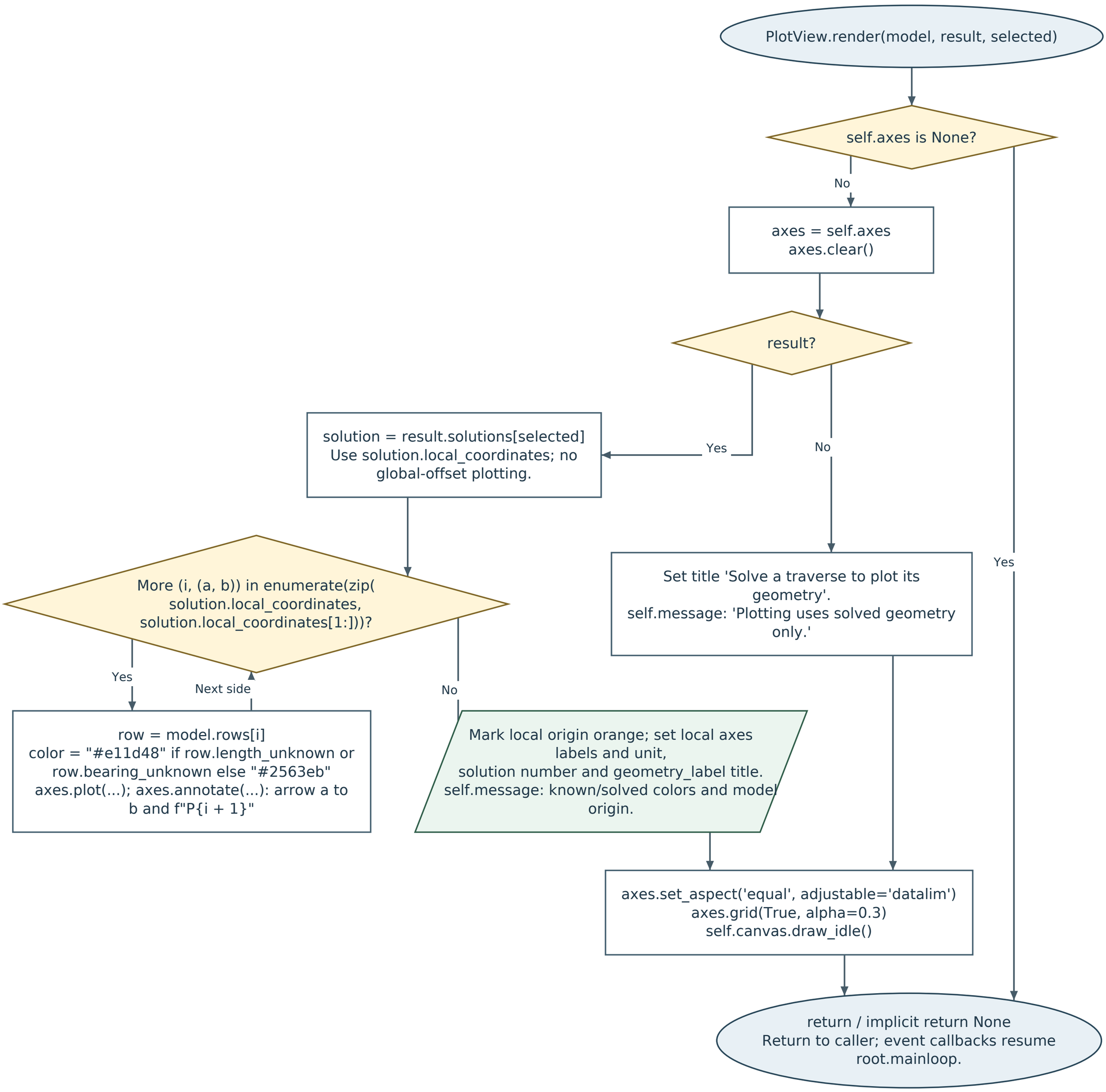
10 Plot backend initialization and recovery

PlotView.__init__() catches Exception during Matplotlib setup; it preserves self.message, removes other widgets, and calls render() (10a).



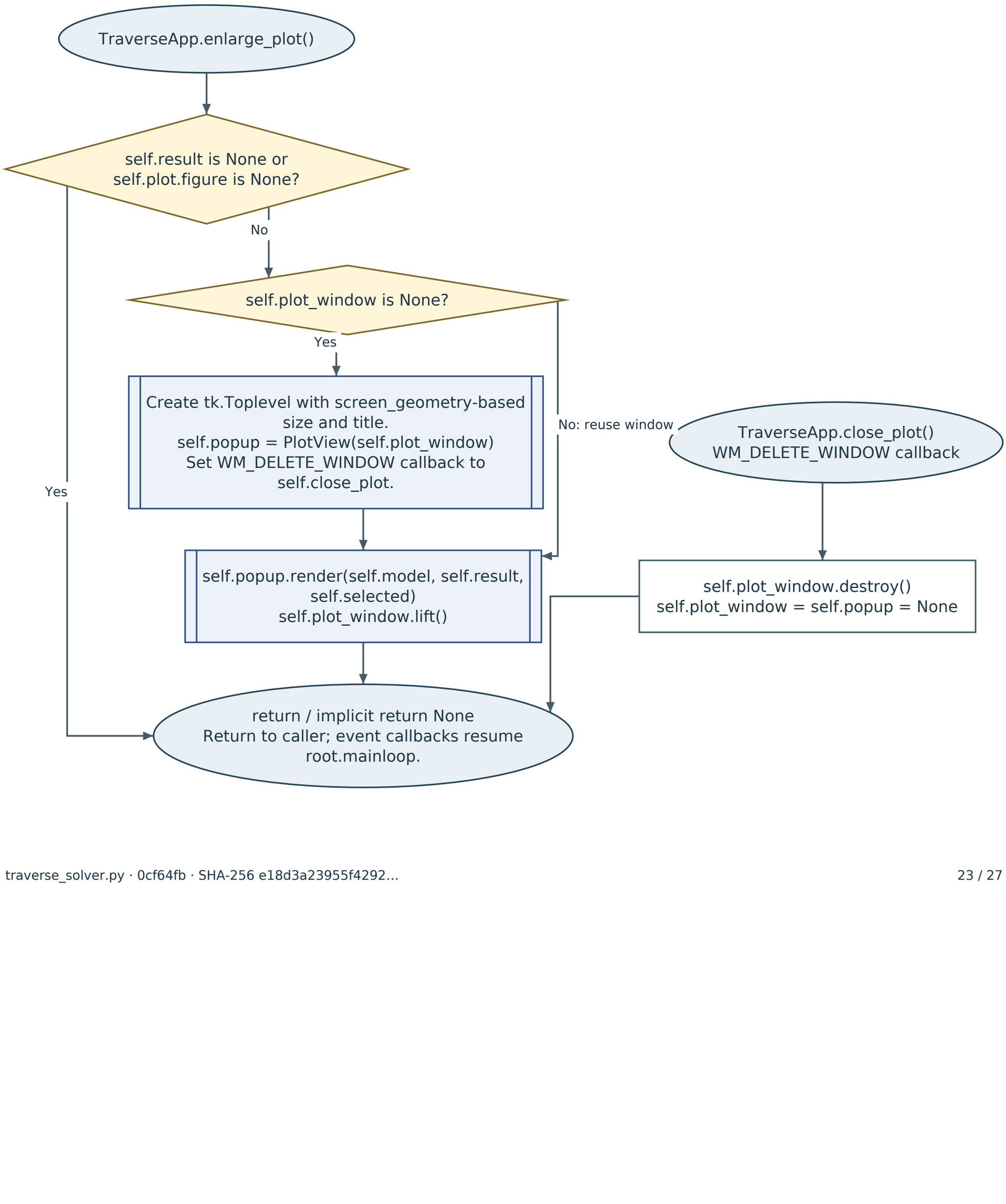
10a Solved plot rendering

PlotView.render() returns immediately without self.axes. Otherwise it draws selected solved geometry or an empty prompt.



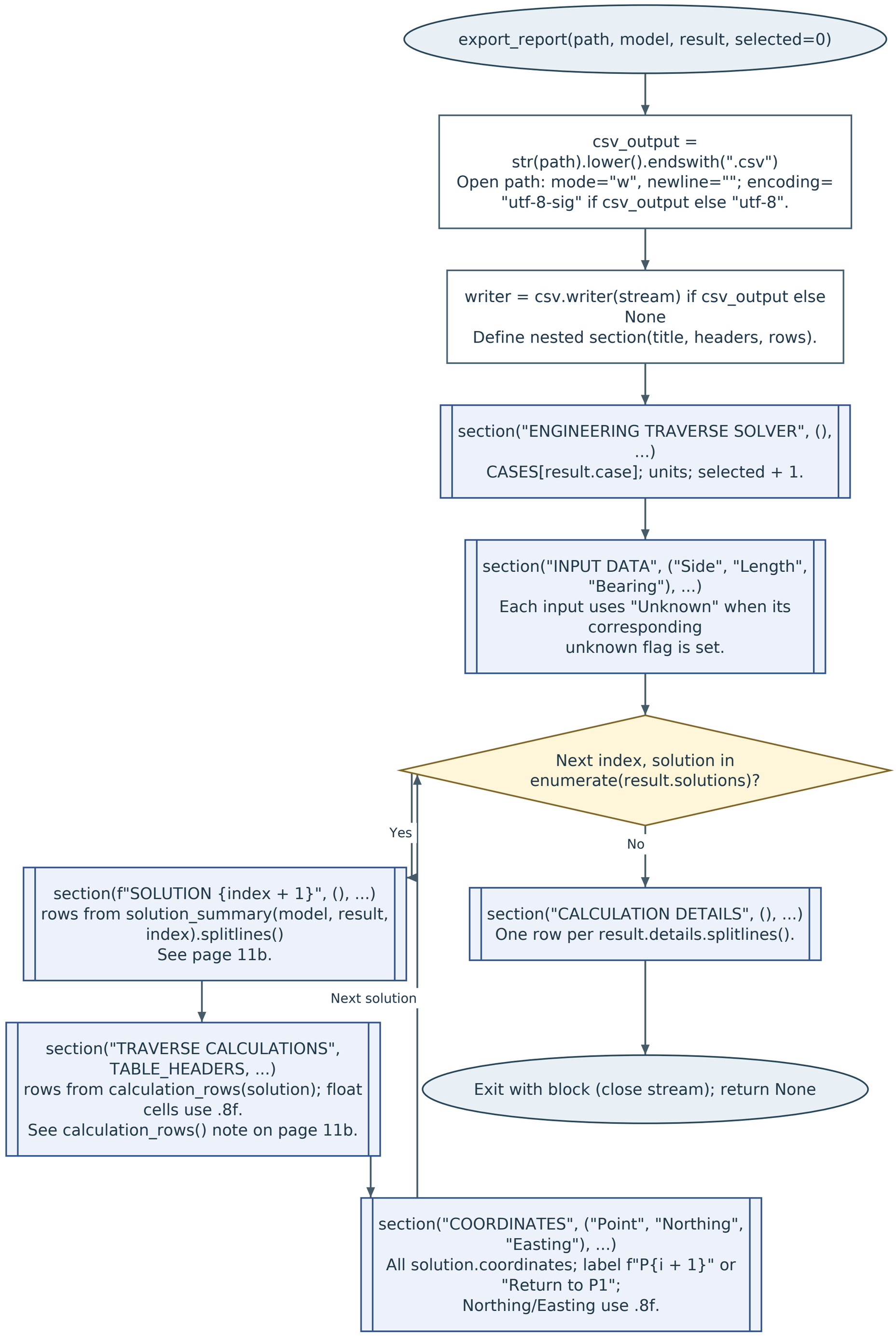
10b Enlarged plot window and close callback

Two callback entries. PlotView initialization/rendering is detailed on 10 and 10a. Root window closure has no custom callback.



11a Export report sections and all-solution loop

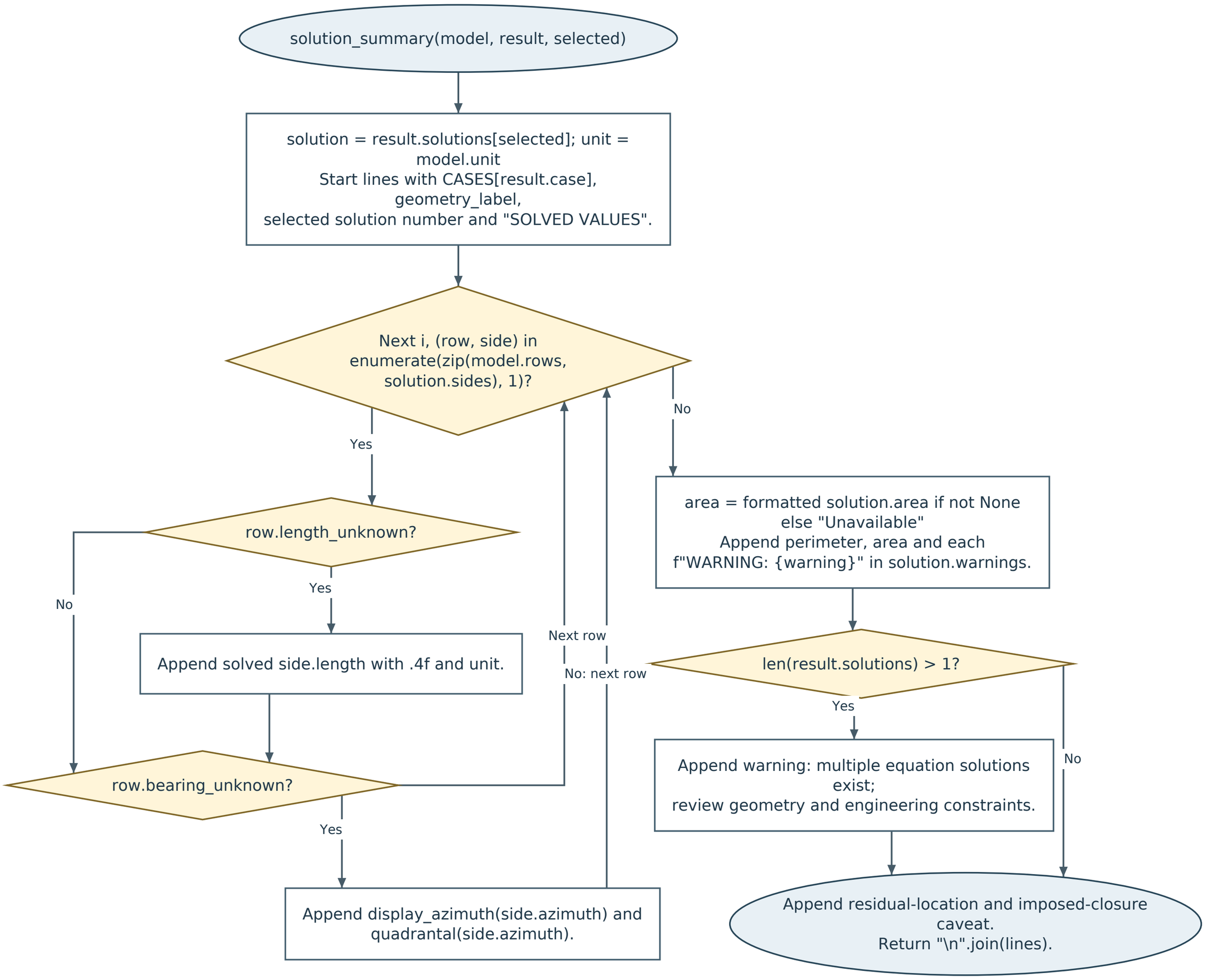
export_report() writes the selected report encoding and all solutions. Nested section() is detailed on 11c.



No local exception handler in export_report() or section().
File / encoding / CSV / formatting errors propagate;
the with block closes an already-open stream on exit.

11b User-facing solution summary

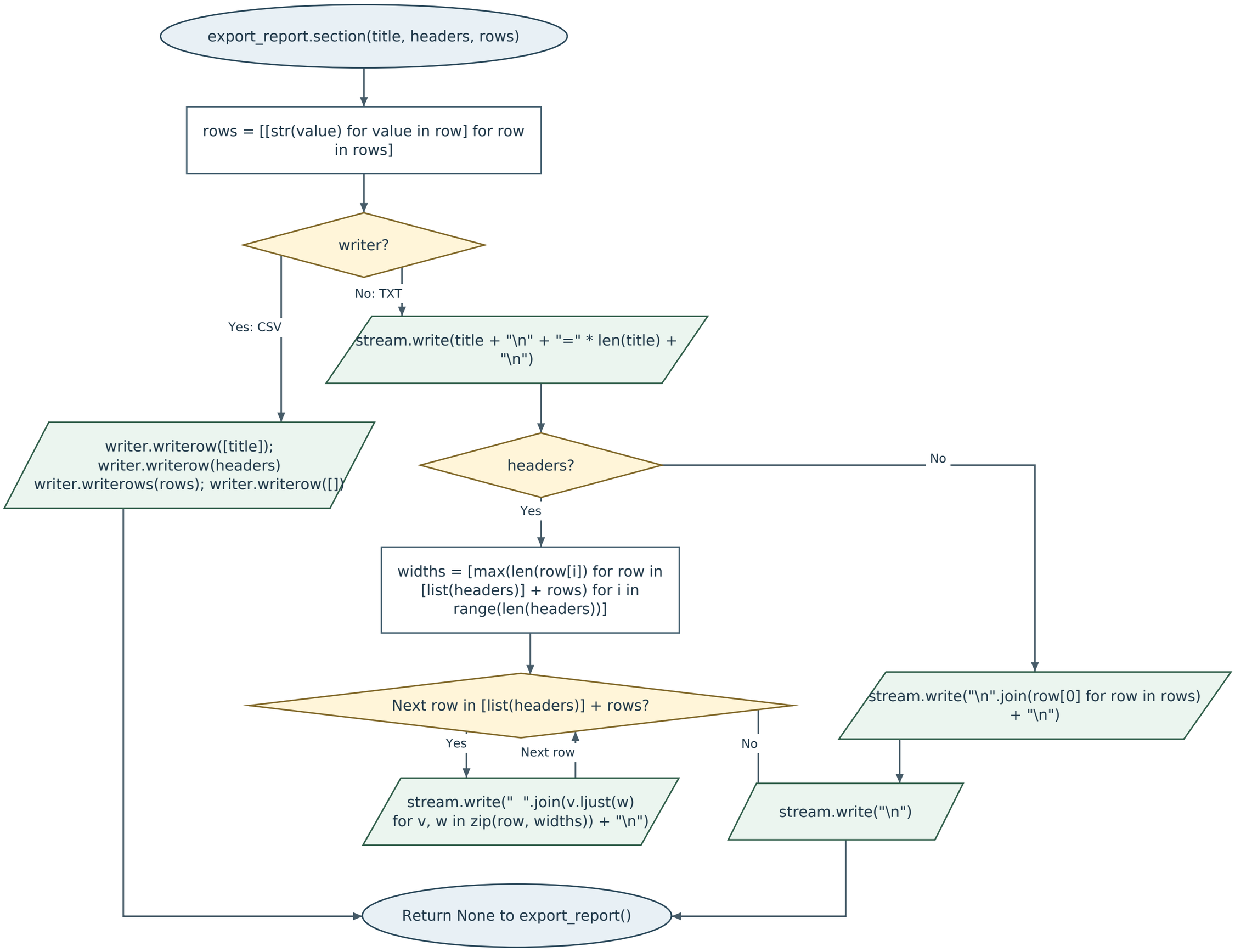
solution_summary() formats the selected solution, omitted values, geometry warnings, and explanatory notes.



calculation_rows(solution): for each i, side, read solution.coordinates[i], [i+1] and solution.components[i]. Yield (i+1, side.length, quadrantal(side.azimuth), side.azimuth, north, east, start_n, start_e, end_n, end_e).

11c CSV or aligned-text section output

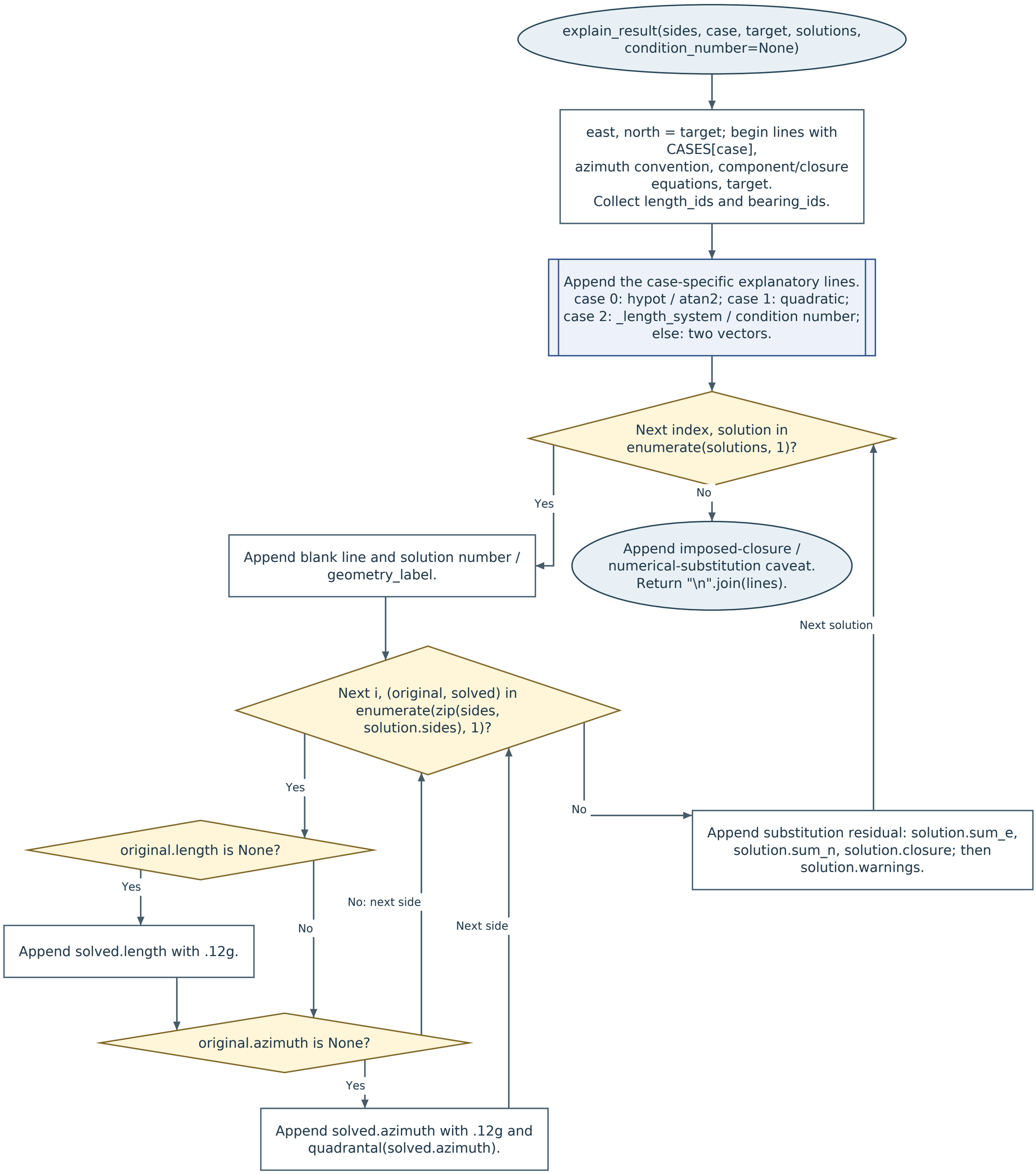
Nested export_report.section() materializes text cells and writes CSV or aligned TXT. Called repeatedly by export_report() (11a).



No local exception handler in export_report() or section().
File / encoding / CSV / formatting errors propagate;
the with block closes an already-open stream on exit.

11d Calculation explanation and solved-value details

explain_result() returns explanatory text for all solutions. Case-specific mathematical branches are expanded on 05–07.



calculation_rows(solution): for each i, side, read
solution.coordinates[i], [i+1] and solution.components[i].
Yield (i+1, side.length, quadrantal(side.azimuth),
side.azimuth, north, east, start_n, start_e, end_n, end_e).